encamina
PIENSA EN COLORES

Modernización de Core Banking API

Manual del taller brownfield | Open Finance

Metodología	Spec Kit (Brownfield) y pruebas primero
Plataforma	C# / .NET 10 / FastEndpoints 8.3.0
Arquitectura	Clean Architecture / Aspire / SQL Server
API	of-sys-acct-core-banking-v1
Regla preservada	Balances 677-680 siguen HTTP 200



ENCAMINA

Octubre 2026

Valencia (sede central)
Av. Jerónimo Roure, 49
46520, Sagunto (Valencia)
Telf. +34 962 698 064

Madrid
C/ O'Donnell, 34
28009, Madrid
Telf. +34 917 893 823

Murcia
C/ del Ángel, 32
30202 Cartagena (Murcia)

Tenerife
Av. Trinidad, 61. Campus Central ULL
38200, S. Cristóbal de La Laguna (Tenerife)

info@encamina.com
www.encamina.com

Modernización de Core Banking API

Índice

1 Objetivo 3
2 Participantes y preparación 3
3 Alcance y entregables 4
4 Las 7 C de la comunicación 4
5 Arquitectura API-Led y nomenclatura 5
6 Caso práctico: Core Banking API existente 5
6.1 Línea base y operaciones 5
6.2 Comportamiento que debe preservarse 6
6.3 Tratamiento de los siete antipatrones 6
7 Organización del repositorio y solución destino 7
7.1 Estructura propuesta de Clean Architecture 7
7.2 Gestión centralizada de versiones NuGet 9
7.3 API Standards: FastEndpoints y documentación 10
7.4 Errores HTTP y excepción heredada 11
7.5 Orquestación local: de Compose a Aspire 12
8 Desarrollo del taller con Spec Kit 13
8.1 Organización de Spec Kit por API 13
8.2 Gobierno y constitución efectiva 13
8.3 Guía para las instrucciones personalizadas de Copilot 14
8.4 Paso 1. Preparar gobierno y capturar la línea base 15
8.5 Paso 2. Redactar la especificación de evolución 15
8.6 Paso 3. Resolver ambigüedades y aprobar compatibilidad 16
8.7 Paso 4. Planificar la migración y descomponer tareas 16
8.8 Paso 5. Agente de modernización y skills necesarias 17
8.9 Paso 6. Implementar mediante pruebas primero 17
8.10 Paso 7. Revisar, validar y aprobar 19
9 Validación y entrega con Jenkins 20
10 Criterios de aceptación 21
11 Resultado esperado 21
12 Referencias oficiales y uso del MCP 22

1 Objetivo

Aplicar Spec Kit a un proyecto existente para evolucionar Core Banking API desde C# y .NET 8 hacia **C# y .NET 10**, con Clean Architecture, gestión centralizada de paquetes NuGet, FastEndpoints 8.3.0, orquestación local con Aspire y desarrollo guiado por pruebas.

El caso continúa el taller greenfield: se trabaja sobre la System API `of-sys-acct-core-banking-v1`, del dominio `account-management` (`acct`), que representa el acceso a capacidades de la plataforma financiera Mambu. En este repositorio, Mambu y la emisión de tokens son simulaciones locales; SQL Server es la fuente persistente. El taller no certifica una integración con un tenant real.

A diferencia del taller greenfield, el resultado incluye la evolución de una implementación ejecutable. No se reescribirá la API sin una línea base: se recuperará su comportamiento, se especificarán los cambios y se modernizará por incrementos verificables.

> **EXCEPCIÓN BROWNFIELD · BALANCES 677–680**
>
> **Excepción obligatoria:** los errores de negocio de balances 677–680 que devuelven HTTP 200 son un bug conocido aceptado como comportamiento correcto para este taller. Se conservarán su estado HTTP, cuerpo, mensajes y tipo de contenido. Esta excepción deberá figurar expresamente en la constitución y en las pruebas; no se corregirá al adoptar Problem Details.

Este documento es el guion del taller. Su elaboración no migra el proyecto, no crea proyectos de código o de tests, no modifica el contrato y no levanta servicios.

2 Participantes y preparación

El taller está dirigido a desarrolladores .NET, responsables funcionales, arquitectos y profesionales de calidad. Antes de comenzar deberán disponer de:

- El repositorio existente y acceso a GitHub Copilot con una revisión identificada de Spec Kit y sus flujos habilitados.
- SDK .NET 10 con revisión exacta acordada y registrada en `global.json`; SDK compatible con .NET 8 mientras se caracteriza el legado.
- Docker Desktop con contenedores Linux x86-64 y memoria suficiente para SQL Server: al menos 2 GB para SQL y 4 GB recomendados para el conjunto.
- Una versión fijada de Aspire y sus herramientas, compatible con .NET 10, y acceso al dashboard local.
- Node.js 20+ y npm para la regresión Newman existente, sin confundirla con una suite automatizada de tests completa.
- Acceso a las fuentes de gobierno aprobadas y a documentación oficial Microsoft mediante su MCP cuando esté disponible.
- Bases aisladas de pruebas, permisos locales adecuados y un plan de preservación de datos. No se utilizarán datos bancarios reales.

Se revisarán el README, los antipatrones, el contrato original, Compose, el Dockerfile, la colección Postman y su guía Newman.

Las credenciales públicas del taller no son credenciales productivas. No se pedirán secretos por chat, no se incluirán en Git, imágenes, informes o telemetría y no se sobrescribirá un entorno local ya configurado.

3 Alcance y entregables

Entregable	Contenido
Línea base brownfield	Inventario de operaciones, comportamiento observado, dependencias, esquema, migraciones y desviaciones del contrato.
Especificación de evolución	Requisitos identificables, compatibilidad, cambios aprobados, excepción HTTP 200 y criterios de aceptación.
Gobierno y decisiones	Constitución efectiva, perfil de API, ADR de arquitectura, paquetes, errores, correlación y orquestación.
Solución modernizada	.NET 10, Domain, Application, Infrastructure y Api con FastEndpoints 8.3.0.
Dependencias reproducibles	Versiones NuGet en `Directory.Packages.props`, SDK fijado y política de restauración verificable.
Entorno local Aspire	AppHost, ServiceDefaults y migrador one-shot con SQL persistente y arranque ordenado.
Pruebas automatizadas	Unitarias, funcionales y de integración derivadas de Postman, más pruebas de arquitectura y riesgos no cubiertos.
Documentación y evidencias	OpenAPI completo del comportamiento aprobado, trazabilidad, resultados Red/Green, regresión y guía de arranque.

Quedan fuera la conexión a Mambu real, la publicación en gestores de APIs y el despliegue productivo. No se crearán Experience o Process APIs, repositorios remotos ni submódulos por el mero hecho de aparecer en el taller greenfield. Si existe un repositorio contractual independiente, su evolución requerirá su propio flujo de aprobación.

4 Las 7 C de la comunicación

Principio	Aplicación brownfield	Comprobación
Claridad	Separar estado actual, objetivo y excepciones.	Cada requisito indica qué cambia y qué se conserva.
Concisión	Referenciar evidencias sin copiar todo el código.	La especificación contiene decisiones, no volcados del repositorio.
Concreción	Precisar rutas, HTTP, payloads, reglas y versiones.	Cada criterio puede traducirse a una aserción.
Corrección	Contrastar negocio, código y documentación oficial.	Ninguna simulación se presenta como regla oficial de Mambu.
Coherencia	Mantener nombres y semántica entre capas y tests.	Un mismo escenario conserva significado y resultado esperado.
Completitud	Incluir seguridad, errores, persistencia y concurrencia.	Se cubren caminos negativos y cambios de estado.

Principio	Aplicación brownfield	Comprobación
Cortesía	Explicar decisiones y riesgos profesionalmente.	Se describen hechos y alternativas sin descalificar el legado.

REGLA DEL 95 % · ACLARACIONES Y CONFIANZA MÍNIMA

Se conservará la regla de aclaraciones del taller: al menos un 95 % de confianza antes de implementar. Una decisión pendiente relevante se preguntará y registrará; no se inventarán requisitos, credenciales, reglas financieras ni URLs de servidores. El porcentaje no sustituye la aprobación ni las pruebas.

5 Arquitectura API-Led y nomenclatura

La API mantiene su responsabilidad **System (sys)**, su dominio `acct` y el identificador `of-sys-acct-core-banking-v1`. Migrar a .NET 10 no implica cambiar la versión pública a `v10`, ni añadir orquestación de inversiones a esta API.

Se conservará la base `/open-finance/system/accounts/v1`. La configuración de versionado de FastEndpoints deberá declarar la versión 1 sin duplicar ni alterar la ruta existente. Un cambio incompatible adicional requerirá una decisión contractual independiente.

El contexto sigue siendo la composición de inversiones del greenfield, pero este taller solo moderniza el acceso a cuentas. Las interfaces de Application serán propias de banca, como `IMambuGateway`, `IAccountRepository` o `IAdjustmentUnitOfWork`; no se incorporarán clientes Northbound ni adaptadores CWMP, ajenos a este dominio.

6 Caso práctico: Core Banking API existente

6.1 Línea base y operaciones

Según el README, la implementación utiliza ASP.NET Core MVC, EF Core 8, SQL Server 2022 Developer, JWT local y un `HttpMessageHandler` que simula Mambu. El diseño no SOLID, los errores de negocio bajo HTTP 200 y Swagger parcial son decisiones deliberadas del material inicial, no recomendaciones de producción.

Método	Ruta relativa a la base	Scope	Resultado esperado
GET	`/customers/{customerId}/accounts`	`accounts.read`	200 con lista, incluida lista vacía.
POST	`/customers/{customerId}/accounts`	`accounts.write`	201 con cuenta `PENDING_APPROVAL`.
GET	`/accounts/{accountId}`	`accounts.read`	200 con referencia.
GET	`/accounts/{accountId}/balances`	`balances.read`	200 con balances o error 677–680.
POST	`/accounts/{accountId}/closure`	`accounts.close`	200 con cuenta `CLOSED`.
POST	`/accounts/{accountId}/balance-adjustments`	`balances.adjust`	201 con movimiento; 202 si continúa en proceso.

También se inventariarán `POST /oauth/token` y `GET /health`. Ambos se migrarán a FastEndpoints; el token seguirá siendo local y la salud comprobará SQL sin exponer credenciales. Swagger y el dashboard son herramientas, no nuevas operaciones bancarias.

6.2 Comportamiento que debe preservarse

- Datos iniciales deterministas: 5 productos, 1000 clientes y 4000 cuentas; 800 cuentas por producto. Las escrituras posteriores pueden aumentar los conteos.
- Migraciones EF versionadas, sin `EnsureCreated`, sin reinserción del seed en cada arranque y sin recreación destructiva de la base.
- Importes `decimal` y persistencia `decimal(28,8)`; en ajustes EUR se rechazan más de dos decimales, no se redondean. No se infiere `valueDate`.
- JWT con validación de firma HS256, issuer, audience y expiración; scopes y visibilidad antes del negocio. Sin token: 401; scope insuficiente: 403; recurso inexistente o ajeno: 404.
- Los cinco scopes, sus 32 subconjuntos y la separación entre `workshop-client` y `workshop-other`. Scope omitido, vacío y desconocido no son equivalentes.
- Ajustes solo en `ACTIVE`, depósitos/retiros según signo, control de fondos y sobregiro y movimientos compensatorios para correcciones. `reason` se audita en SQL, no se envía al mock Mambu.
- Clave idempotente UUID v4 única por consumidor, payload normalizado, 409 ante payload distinto, 202 durante `Processing` y replay del resultado completado sin doble posting.
- Atomicidad entre balances, movimiento e idempotencia; SQL parametrizado, bloqueos, claves únicas, FK y `rowversion` se preservan o sustituyen por controles equivalentes probados.
- Cierre de `ACTIVE/APPROVED` con balances cero y sin retenciones; `CLOSED` repetido devuelve 200; otras precondiciones/transiciones devuelven 409.
- La API sigue limitada a `Workshop/Development`; Mambu continúa interceptado localmente y nunca recibe credenciales sintéticas en un tenant real.

`ExpiresAt` seguirá siendo metadata, no una limpieza ya implementada; un `Processing` abandonado seguirá requiriendo reconciliación manual mientras no se apruebe otro requisito. Estos límites se documentarán, no se resolverán mediante cambios funcionales implícitos.

6.3 Tratamiento de los siete antipatrones

Antipatrón del documento original	Trabajo del taller	Evidencia
1. Controlador con múltiples responsabilidades	Sustituir por endpoints delgados, casos de uso y adaptadores.	Tests por operación y ausencia de negocio en Api.
2. Negocio acoplado a persistencia/proveedor	Invertir dependencias mediante puertos de Application e implementaciones Infrastructure.	Tests de arquitectura y casos de uso sin SQL/HTTP real.
3. Reglas rígidas y literales dispersos	Encapsular estados, moneda, transiciones y configuración del canal mock.	Pruebas de políticas y configuración; sin convertir reglas financieras en configuración arbitraria.
4. Invariantes fuera de la entidad	Proteger mutaciones y transiciones con entidades/value objects del dominio.	Tests de fondos, escala, retenciones, cierres y estados inválidos.
5. Fallos de negocio como HTTP 200	**No corregir: excepción constitucional obligatoria.**	Pruebas de regresión exacta 677–680 y documentación OpenAPI de variantes.

Antipatrón del documento original	Trabajo del taller	Evidencia
6. OpenAPI incompleto	Describir operaciones, propiedades, restricciones, scopes, errores y ejemplos reales.	Validación contractual y comparación de documento con ejecución.
7. Replay con correlación antigua	Separar resultado estable de operación y metadata de la petición actual.	Replay con distinta correlación, sin mutación ni doble posting.

La instrucción de corregir los antipatrones se aplica a 1, 2, 3, 4, 6 y 7; el 5 constituye la única excepción explícita de esta lista. No se eliminarán controles útiles por estar ubicados hoy en un controlador.

7 Organización del repositorio y solución destino

Se trabajará sobre el repositorio existente. Si se aloja en GitLab, se utilizarán merge requests sin duplicar proyectos ni crear grupos. Jenkins seguirá siendo el orquestador propuesto, separado del código del servicio. No se generará GitLab CI/CD como efecto colateral.

7.1 Estructura propuesta de Clean Architecture

La propuesta aplica los principios de inversión de dependencias y separación de responsabilidades de Microsoft. Es una decisión para este caso, no una plantilla impuesta por Microsoft ni un motivo para introducir microservicios.

```
of-sys-acct-core-banking/
  CoreBanking.slnx
  global.json
  Directory.Build.props
  Directory.Packages.props
  NuGet.Config
  .specify/
    memory/constitution.md
  governance/
    api-profile.md
  specs/
    001-modernize-core-banking/
      spec.md
      plan.md
      research.md
      data-model.md
      quickstart.md
      tasks.md
      contracts/
  src/
    CoreBanking.Domain/
      Accounts/
      Adjustments/
      Errors/
    CoreBanking.Application/
      Accounts/
      Balances/
      Adjustments/
      Abstractions/
      DependencyInjection.cs
    CoreBanking.Infrastructure/
      Persistence/
        Configurations/
        Migrations/
      Mambu/
      Identity/
      DependencyInjection.cs
    CoreBanking.Api/
      Endpoints/
        Accounts/
        Balances/
        Tokens/
        Health/
      Contracts/
      ErrorHandling/
      Program.cs
    CoreBanking.MigrationService/
    CoreBanking.AppHost/
    CoreBanking.ServiceDefaults/
  tests/
    CoreBanking.UnitTests/
    CoreBanking.FunctionalTests/
    CoreBanking.IntegrationTests/
  docs/
  postman/
  compose.yaml
  Dockerfile
```

Esta estructura es un objetivo del taller, no una descripción de archivos ya existentes. Se conservará `CoreBanking.Api` como proyecto de entrada para reducir cambios de identidad. Las carpetas se crearán cuando exista código que las justifique; no se exigirá un repositorio genérico, mediador o bus de eventos sin necesidad demostrada.

Proyecto	Responsabilidad	Referencias permitidas
Domain	Entidades, value objects, invariantes, servicios y errores de dominio.	Ninguna otra capa; sin ASP.NET Core, EF Core, HTTP ni paquetes de infraestructura.
Application	Casos de uso de las seis operaciones, puertos, DTOs de aplicación y validación de casos de uso.	Domain; nunca Infrastructure o Api.
Infrastructure	EF Core, SQL, transacciones, idempotencia persistente, cliente/mock Mambu y adaptadores técnicos.	Application y Domain.
Api	FastEndpoints, DTOs HTTP, validadores integrados, autenticación/autorización HTTP, middleware, errores y composición.	Application; Infrastructure solo desde la raíz de composición.
MigrationService	Ejecutar migraciones pendientes y terminar con código explícito.	Infrastructure y utilidades técnicas necesarias.
AppHost	Modelar recursos locales y dependencias de arranque.	Proyectos ejecutables como recursos; no gobierna reglas bancarias.
ServiceDefaults	Observabilidad, configuración transversal y registro de comprobaciones de salud.	Bibliotecas técnicas; no Domain ni negocio.

Las flechas de dependencia de compilación son `Application -> Domain`, `Infrastructure -> Application/Domain` y `Api -> Application`. La referencia `Api -> Infrastructure` se limita al composition root para registrar adaptadores. Domain no depende de Application. Separar proyectos sin respetar estas direcciones no satisface Clean Architecture.

Se registrarán servicios con extensiones por capa: `AddApplication()` y `AddInfrastructure(configuration)`. Domain no necesita una extensión de DI. Las extensiones no ocultarán un service locator ni resolverán dependencias desde el dominio. Los endpoints recibirán casos de uso, no `BankingDbContext` ni el cliente Mambu concreto.

Las entidades de dominio, modelos EF, DTOs de aplicación y contratos HTTP tendrán límites explícitos. Las configuraciones EF y el mapeo al esquema existente vivirán en Infrastructure; mover tipos no autoriza a perder datos, cambiar importes o regenerar la migración inicial.

7.2 Gestión centralizada de versiones NuGet

Todas las versiones de paquetes NuGet se administrarán en un único `Directory.Packages.props` en la raíz. Se activará `ManagePackageVersionsCentrally` y se deshabilitará `CentralPackageVersionOverrideEnabled`. No habrá archivos centrales anidados que creen catálogos divergentes ni exclusiones de CPM por proyecto.

Ejemplo normativo para las dos versiones exigidas:

```
<Project>
  <PropertyGroup>
    <ManagePackageVersionsCentrally>true</ManagePackageVersionsCentrally>
    <CentralPackageVersionOverrideEnabled>false</CentralPackageVersionOverrideEnabled>
  </PropertyGroup>
  <ItemGroup>
    <PackageVersion Include="FastEndpoints" Version="8.3.0" />
    <PackageVersion Include="FastEndpoints.Swagger" Version="8.3.0" />
  </ItemGroup>
</Project>
```

Cada proyecto solo declarará el identificador del paquete, nunca su versión:

```
<PackageReference Include="FastEndpoints" />
<PackageReference Include="FastEndpoints.Swagger" />
```

Se permiten metadatos de consumo como `PrivateAssets` para herramientas de diseño; se prohíben `Version`, el elemento hijo `Version` y `VersionOverride` en `PackageReference`. EF Core, JwtBearer, Aspire, OpenTelemetry, herramientas de test y cualquier otra dependencia directa deberán tener su entrada central con versión exacta compatible. No se afirmará que deben compartir el mismo número mayor entre productos distintos.

La versión del SDK .NET corresponde a `global.json`; el TFM `net10.0` se acordará en las propiedades de compilación compartidas. Las versiones de SDK MSBuild, como `Aspire.AppHost.Sdk` cuando se utilice, se fijarán mediante el mecanismo de SDK correspondiente, no mediante `PackageVersion`. La versión de la CLI y las herramientas locales también se fijará en su configuración propia: CPM no controla herramientas, imágenes o SDKs.

Se acordará una política de `packages.lock.json` y restauración en modo bloqueado para los proyectos que lo utilicen, además de fuentes NuGet y source mapping cuando proceda. Una comprobación automatizada fallará ante versiones locales o paquetes centrales ausentes; restaurar correctamente no basta para detectar todas las evasiones de esta política.

7.3 API Standards: FastEndpoints y documentación

- **FastEndpoints 8.3.0 y FastEndpoints.Swagger 8.3.0 son obligatorios**, administrados centralmente. No se sustituirán silenciosamente por versiones más recientes.
- Cada operación HTTP tendrá una clase propia derivada de `Endpoint<TRequest, TResponse>` o de una base apropiada sin request o sin response. Esto incluye cuentas, balances, token mock y salud pública.
- `Configure()` declarará verbo, ruta, versión y autorización; las excepciones anónimas, como el token local y la salud según la política aprobada, serán explícitas. Se conservarán las rutas públicas actuales.
- `HandleAsync` delegará en casos de uso y propagará su `CancellationToken` hasta EF Core y el adaptador Mambu. No contendrá reglas de fondos, cierres o idempotencia.
- Los DTOs HTTP serán distintos de entidades de dominio y persistencia. Los validadores integrados con FastEndpoints comprobarán forma y restricciones de entrada; las invariantes financieras seguirán en Domain.
- Se prohíben MVC controllers y mappings Minimal API (`MapGet`, `MapPost`, `MapGroup`, etc.) para operaciones HTTP. Al completar la migración no quedarán `AddControllers` ni `MapControllers`.
- La documentación usará `SwaggerDocument(...)` y `UseSwaggerGen()` de FastEndpoints.Swagger. Se retirará el registro Swagger de Swashbuckle anterior sin mantener dos generadores en paralelo.

Registro condicionado: antes de registrar servicios se determinará si el conjunto de assemblies de endpoints seleccionado contiene declaraciones concretas. Se utilizará esa misma selección para el descubrimiento y una única decisión coherente para `AddFastEndpoints`, `SwaggerDocument(...)`, `UseFastEndpoints` y `UseSwaggerGen()`.

Si no hay declaraciones de endpoints, la aplicación arrancará sin registrar FastEndpoints y sin definir ni exponer ningún documento o UI Swagger. No basta con esconder la UI, ni con comprobar si ya llegaron peticiones. Se probarán un host con endpoints y otro con un conjunto vacío, verificando servicios, rutas y ausencia de documento Swagger.

ServiceDefaults no deberá introducir `MapDefaultEndpoints()` u otros mappings Minimal API para esquivar esta regla. Las comprobaciones de salud pueden registrarse como servicios y exponerse mediante endpoints FastEndpoints que consulten `HealthCheckService`; cualquier endpoint técnico adicional deberá estar especificado y autorizado.

7.4 Errores HTTP y excepción heredada

Para los errores HTTP se adoptará **Problem Details conforme a RFC 9457, que sustituye RFC 7807**, con `application/problem+json`. Cuando se incluya `status`, coincidirá con el estado HTTP real; se definirán `type`, `title`, `detail` e `instance` según corresponda, además de extensiones seguras como `correlationId` y códigos de negocio aplicables. No se expondrán stack traces, SQL, tokens o secretos.

Se integrarán `AddProblemDetails`, el manejo global de excepciones y el escritor de respuestas que corresponda con FastEndpoints 8.3.0. No se asumirá que el formato de validación predeterminado del framework ya cumple la política: validadores, autenticación, autorización, rate limiting, errores no controlados y respuestas generadas por middleware deberán probarse expresamente.

Se mantendrá el catálogo observado de estados para operaciones bancarias: 400 para entrada inválida, 401/403/404 para acceso, 409 para conflictos, 429 para limitación y 500/502/503/504 para los escenarios técnicos caracterizados. La política no convertirá indiscriminadamente todas las excepciones a un solo estado.

Escenario de consulta de balances	HTTP y contenido que se deben conservar
677, consulta bloqueada	200, `application/json`, `{"errorCode":677,"message":"Balance is not posible"}`.
678, cuenta bloqueada	200, `application/json`, mensaje `Balance inquiry is not allowed for a locked account`.
679, pendiente de aprobación	200, `application/json`, mensaje `Balance is unavailable while the account is pending approval`.
680, información ausente	200, `application/json`, mensaje `Required balance information is missing`.

Las cuatro respuestas contienen únicamente el error de negocio acordado, sin balances parciales; no se envolverán en Problem Details, no se cambiarán a 4xx/5xx y no se corregirá la ortografía de 677. Se conservará la prioridad: consulta bloqueada, `LOCKED`, `PENDING_APPROVAL` y después ausencia de balances. Primero se validará autenticación, scope y visibilidad.

OpenAPI describirá el 200 de balances con `oneOf` entre la respuesta de balances y la de error de negocio, con códigos, mensajes y ejemplos de las cuatro variantes. La observabilidad distinguirá esos errores mediante métricas o eventos de negocio sin falsear su estado HTTP.

El endpoint OAuth mock conserva los errores de protocolo `invalid_client`, `invalid_scope` y `unsupported_grant_type` con sus estados caracterizados. Se documentará esta excepción de formato OAuth frente a Problem Details para no romper el protocolo al aplicar una política genérica.

Para corregir la correlación del replay se aprobará esta política: el resultado estable de negocio y el estado se conservan; el header y `correlationId` del Problem Details representan la petición actual; una extensión `originalCorrelationId` identifica la operación original. No se sobrescribirá el registro original ni se recalculará el

posting. Este cambio de metadata se registrará como cambio aprobado respecto del replay byte a byte anterior y se comprobará con la misma clave y dos correlaciones distintas.

7.5 Orquestación local: de Compose a Aspire

AppHost será el punto de entrada local y describirá recursos, conexiones y esperas; no ejecutará reglas bancarias. Se conservarán Compose y Dockerfile como alternativa verificable mientras dure la transición. No se arrancarán ambos sobre los mismos puertos o datos simultáneamente.

Configuración actual	Configuración objetivo en Aspire	Comprobación
`sqlserver`, imagen `2022-CU18-ubuntu-22.04`	Recurso SQL Server con `Aspire.Hosting.SqlServer`, imagen/tag explícitos, edición Developer y aceptación de licencia.	No cambiar de motor ni aceptar implícitamente el tag predeterminado de Aspire.
`SQL_PASSWORD` y usuario local `sa`	Parámetro secreto del AppHost o user-secrets, suministrado a SQL y a los consumidores.	Sin secretos en código/dashboard exportado; siguen siendo credenciales locales, no productivas.
Volumen `sql-data`	`WithDataVolume` con nombre explícito y estrategia de volumen acordada.	Persistencia tras reinicio; nunca asumir que el nombre efectivo de Compose coincide con el de Aspire.
Base `CoreBankingWorkshop`	Base lógica con nombre de recurso `Banking` y nombre físico conservado.	`WithReference` inyecta `ConnectionStrings__Banking` en API y migrador.
Healthcheck SQL con `SELECT 1`	Healthcheck SQL de la integración y espera `WaitFor` del migrador.	No basta con que el contenedor esté iniciado.
`migrate --migrate-only`, `restart: no`	Proyecto `CoreBanking.MigrationService` one-shot, sin servidor HTTP, que migra y termina.	Código 0 solo si terminó correctamente; fallo impide arrancar API.
API depende de migración completada	API con referencia a `Banking` y `WaitForCompletion` del migrador.	Cadena SQL saludable → migración completada → API; sin migraciones desde cada réplica.
`ASPNETCORE_ENVIRONMENT=Workshop`	Variable explícita para la API; configuración equivalente para el migrador.	No arrancar el mock en Production por defecto.
SigningKey, ClientSecret y RateLimit	Parámetros/configuración para `Workshop__SigningKey`, `Workshop__ClientSecret` y `Workshop__RateLimit`.	Sin confiar en carga automática de `.env`; validar mínimos y precedencia.
API host 5000 y SQL host 14333, loopback	Endpoints locales configurables, conservando esos puertos solo cuando estén libres.	Sin detener servicios ajenos; actualizar `origin` de Postman si cambia la API.
Contenedor API en 8080, usuario `app`	Ejecución local por proyecto con .NET 10; perfil contenedorizado conserva puerto interno y usuario no root.	Diferenciar puerto del proceso, proxy Aspire y puerto publicado.

Configuración actual	Configuración objetivo en Aspire	Comprobación
Dockerfile multietapa .NET 8	SDK/runtime .NET 10 y copia de catálogo NuGet y proyectos referenciados antes de restaurar.	Build reproducible con nuevas capas; secretos y `bin`/`obj` fuera del contexto útil.

Se documentarán `WithReference`, `WaitFor` y `WaitForCompletion` como responsabilidades distintas: pasar una conexión no equivale a esperar disponibilidad o finalización. Las APIs concretas se contrastarán con la versión de Aspire fijada antes de escribir AppHost.

El volumen existente no se renombrará ni conectará a dos servidores sin aprobación. Por defecto, Aspire usará almacenamiento separado; la adopción de datos anteriores requerirá respaldo y restauración o una estrategia revisada. Las pruebas usarán bases únicas por ejecución y nunca borrarán bases o volúmenes del usuario.

El migrador mantendrá historial, snapshot y seed. Al mover las migraciones a Infrastructure se verificará la base existente con `__EFMigrationsHistory`, la ausencia de cambios inesperados y la aplicación sobre una base vacía. Un cambio de namespace o assembly no autoriza a emitir una migración destructiva.

ServiceDefaults aportará logs, métricas y trazas con OpenTelemetry donde corresponda. No añadirá reintentos automáticos a depósitos/retiros sin evaluar la idempotencia del proveedor: la deduplicación local no garantiza que un tercero no haya aplicado una operación cuyo resultado se perdió.

8 Desarrollo del taller con Spec Kit

8.1 Organización de Spec Kit por API

Se incorporará Spec Kit al repositorio existente sin reemplazar sus fuentes. La constitución efectiva estará en `.specify/memory/constitution.md` y cada evolución tendrá sus artefactos en `specs/`. Se revisarán los archivos existentes antes de inicializar; no se utilizará una opción de sobrescritura forzada como procedimiento normal.

La guía oficial de proyectos existentes deberá consultarse para la revisión elegida. Los nombres de invocación dependen de la versión e integración: pueden encontrarse formas `/speckit-specify` o `/speckit.specify`. El equipo comprobará las disponibles en VS Code antes del taller; los nombres de flujo usados a continuación no son comandos de terminal ni garantizan que una integración antigua los exponga.

8.2 Gobierno y constitución efectiva

Se mantiene el modelo del greenfield: constitución corporativa, constitución del dominio `account-management`, perfil de especialización de la API y una constitución efectiva generada con fuentes/revisiones trazables. No se presupone herencia automática de archivos por pertenecer a un grupo GitLab.

No se reutilizará sin adaptación la prohibición greenfield de generar backend. Las fuentes de gobierno deberán aprobar un alcance brownfield de implementación antes de componer la constitución. Un perfil local no puede rebajar una prohibición corporativa; el conflicto debe resolverse en su fuente o mediante una excepción aprobada. La generación de este documento no implementa el mecanismo de composición.

Los siguientes principios serán requisitos obligatorios de la constitución efectiva:

Principio	Obligación que debe recoger
I. Evolución basada en evidencia	Caracterizar el legado y conservar compatibilidad salvo cambios aprobados.
II. Backend .NET 10	C#, TFM `net10.0`, SDK y herramientas fijados, migración incremental.

Principio	Obligación que debe recoger
III. Clean Architecture y SOLID	Dependencias hacia dentro, dominio encapsulado, puertos y endpoints sin negocio.
IV. Tests primero	Escribir y ejecutar pruebas antes de implementar cada comportamiento o corrección; conservar evidencias Red/Green/Refactor.
V. Persistencia y seguridad	Preservar datos, atomicidad, idempotencia, autorización y secretos; Mambu permanece mock.
VI. Errores y compatibilidad	RFC 9457 para errores HTTP y excepción obligatoria del bug 677–680 bajo HTTP 200.
VII. NuGet y API Standards	CPM estricto, FastEndpoints y Swagger 8.3.0, una clase por operación, sin MVC/Minimal APIs y registro condicionado a declaraciones.
VIII. Entorno reproducible	Aspire con SQL saludable, migrador one-shot, API posterior y pruebas aisladas.

Texto obligatorio del principio VI que se trasladará a la constitución, en inglés:

> Known bug accepted as correct workshop behavior: balance inquiry business errors 677, 678, 679 and 680 MUST continue to return HTTP 200 with application/json and the existing errorCode/message payloads. They MUST NOT be converted to RFC 9457 Problem Details or to non-200 HTTP status codes. Preserve their messages, evaluation precedence and authorization checks. Fixing other anti-patterns does not authorize fixing this bug. Any future correction requires a separately approved specification and compatibility decision.

Esta cláusula prevalece dentro del alcance acordado sobre la instrucción general de corregir antipatrones. Los cambios de gobierno se realizarán en la fuente o perfil correspondientes y se regenerará la constitución; no se editará silenciosamente el documento generado.

8.3 Guía para las instrucciones personalizadas de Copilot

Como en el greenfield, el equipo preparará pautas generales breves y reglas por ámbito en inglés. Este guion no crea sus archivos.

Ámbito	Pautas
Repositorio	Consultar constitución, perfil, especificación y línea base; no inventar requisitos ni sobrescribir trabajo.
Domain/Application	Dependencias permitidas, invariantes, puertos, cancelación y pruebas unitarias antes de implementación.
Infrastructure	EF/SQL, migraciones existentes, mock Mambu, transacciones y tests de integración reales.
Api	FastEndpoints 8.3.0, DTOs separados, validadores, RFC 9457 y excepción HTTP 200.
Paquetes y AppHost	CPM, versiones fijadas, secretos, persistencia y orden de arranque Aspire.

Ámbito	Pautas
Tests	Trazabilidad Postman, aislamiento y evidencia de fallo previo; no falsear aprobación mediante skipped tests.

Las instrucciones no sustituirán la constitución ni autorizarán cambios funcionales. Se comprobará su aplicación real en VS Code; las reglas verificables seguirán siendo controles automatizados.

8.4 Paso 1. Preparar gobierno y capturar la línea base

1. Identificar revisión Git, cambios locales y configuración actual sin revelar `.env`.
2. Revisar README, antipatrones, código, contrato, migraciones y Postman; registrar discrepancias entre intención, implementación y ejemplos.
3. Ejecutar el arranque y la regresión actuales sobre datos aislados. El ejecutor Newman existente usa Compose y puede detener temporalmente la API principal: reservar la ventana y no ejecutarlo contra Aspire como si ya estuviera adaptado.
4. Conservar informes, conteos del seed, rutas, status, payloads, headers y efectos SQL; los informes previos no prueban una ejecución nueva.
5. Aprobar gobierno brownfield y generar la constitución efectiva con la excepción HTTP 200 antes de planificar cambios.

La colección cuenta actualmente con 409 solicitudes; el README informa de 408 ejecutadas, excluyendo la saturación manual 429. Se verificará ese inventario en la revisión elegida; no se presentarán 409 solicitudes como 409 tests con aserciones ya existentes.

8.5 Paso 2. Redactar la especificación de evolución

El flujo de especificación recogerá objetivos y comportamiento observable, sin limitarse a pedir «aplicar Clean Architecture». Se distinguirán requisitos conservados, cambios aprobados y restricciones técnicas.

Identificador propuesto	Requisito verificable
BR-001	Compilar y ejecutar el backend sobre .NET 10.
BR-002	Centralizar todas las versiones NuGet sin versiones locales u overrides.
BR-003	Respetar límites Domain/Application/Infrastructure/Api y corregir antipatrones estructurales.
BR-004	Migrar las seis operaciones, OAuth mock y salud a FastEndpoints 8.3.0.
BR-005	Arrancar desde Aspire con SQL persistente y migración one-shot bloqueante.
BR-006	Implementar pruebas unitarias, funcionales e integración antes del código nuevo.
BR-007	Aplicar RFC 9457 y preservar exactamente la excepción HTTP 200 para 677–680.
BR-008	Corregir correlación de replay según la política aprobada sin doble posting.

Identificador propuesto	Requisito verificable
BR-009	Completar OpenAPI, scopes, variantes, restricciones y ejemplos.
BR-010	Conservar reglas financieras, seguridad, seed, esquema e idempotencia.
BR-011	No registrar FastEndpoints ni definir/exponer Swagger si no hay endpoints declarados.

Por operación se documentarán entradas, salidas, errores, autorización, escenarios del mock, efectos SQL y criterios Given/When/Then. Cada requisito se relacionará con escenarios Postman y con pruebas destino; un ejemplo guardado no se considerará una especificación exhaustiva de todos los campos.

8.6 Paso 3. Resolver ambigüedades y aprobar compatibilidad

Antes de implementar se aprobarán las revisiones exactas de SDK, EF Core, Aspire y runner de tests; la compatibilidad con los dos paquetes FastEndpoints 8.3.0; la política de versionado sin cambios de rutas; el formato de errores OAuth; la metadata de replay; los puertos y la estrategia de datos.

Las versiones restantes se seleccionarán tras comprobar compatibilidad y soporte; no se rellenarán con números inventados. Si una restricción de paquete impide restaurar, se registrará el bloqueo en lugar de relajar la versión obligatoria.

Se revisará especialmente la desviación entre el contrato original y el 200 con error de balances. El contrato de evolución documentará la excepción sin «arreglar» a la vez el comportamiento. La adopción de modelos compartidos del greenfield requerirá confirmar repositorios y revisiones; no se supondrá que esos submódulos existen en este proyecto de implementación.

8.7 Paso 4. Planificar la migración y descomponer tareas

El flujo de planificación producirá decisiones técnicas, estructura, estrategia de datos y plan de pruebas. El flujo de tareas ordenará cada incremento como **prueba → fallo esperado → implementación mínima → verde → refactorización → regresión**.

Secuencia propuesta:

1. Añadir pruebas de caracterización del legado y la matriz de trazabilidad, sin cambiar sus resultados.
2. Establecer controles de paquetes y tests de arquitectura; preparar CPM y migrar SDK/TFM/dependencias a .NET 10 como incremento técnico aislado.
3. Extraer Domain y Application por caso de uso, probando antes invariantes y puertos; mover persistencia y adaptadores a Infrastructure sin alterar el esquema.
4. Preparar tests HTTP y sustituir cada operación MVC por FastEndpoints, sin dos handlers activos para la misma ruta. La convivencia temporal se limita a la migración; la entrega final no conserva controllers.
5. Probar y corregir errores, correlación y OpenAPI; demostrar que 677–680 permanecen intactos.
6. Probar recursos y dependencias de Aspire, introducir migrador y ServiceDefaults y actualizar el build Docker para .NET 10 y CPM.
7. Ejecutar las tres suites, la regresión de compatibilidad y la revisión de evidencias antes de retirar rutas/configuración obsoletas.

No se ejecutarán extracciones masivas y cambios de HTTP en un único paso opaco. La revisión de consistencia de Spec Kit comprobará constitución, especificación, plan y tareas antes del flujo de implementación.

8.8 Paso 5. Agente de modernización y skills necesarias

El equipo puede preparar un agente `Modernize Core Banking` y skills reutilizables con nombres como los siguientes. Se trata de recursos propuestos, no instalados por este documento.

Skill propuesta	Resultado y control
`characterize-core-banking`	Inventario del comportamiento existente y matriz Postman/requisito/test, sin corregir la excepción.
`plan-dotnet-modernization`	Plan .NET 10, CPM, Clean Architecture, compatibilidad y preservación de datos.
`test-first-core-banking`	Pruebas unitarias, funcionales e integración y evidencias previas a implementación.
`migrate-fastendpoints`	Una operación por clase, validación, autorización y registro/documentación condicionados.
`orchestrate-aspire-local`	SQL, migrador, API y observabilidad con arranque, secretos y volumen comprobados.
`verify-brownfield-regression`	Cobertura contractual/SQL, antipatrones, excepción HTTP 200 y documentación final.

Cada skill declarará entradas, precondiciones, herramientas reales, procedimiento, límites y evidencias. El agente consultará documentación Microsoft por MCP para arquitectura, NuGet, errores, EF y Aspire; utilizará documentación del fabricante para FastEndpoints y Spec Kit. No dispondrá de permisos remotos o productivos por el mero hecho de nombrar herramientas.

Se distribuirán recursos versionados desde el gobierno del dominio cuando exista ese mecanismo, comprobando que VS Code realmente los descubre. No se copiarán reglas divergentes por API. El agente no cambiará gobierno compartido, conectará Mambu real ni desplegará servicios sin autorización separada.

8.9 Paso 6. Implementar mediante pruebas primero

8.9.1 Convertir Postman en pruebas ejecutables

La colección es el inventario inicial, no una suite completa. Sus scripts post-response capturan tokens/IDs; el ejecutor compara principalmente HTTP y códigos de negocio. Se derivarán aserciones de cuerpo, headers, autorización y estado persistido, sin copiar ciegamente IDs dinámicos o ejemplos incompletos.

Carpeta de la colección	Solicitudes actuales	Pruebas derivadas
`00 - Workshop mock tokens`	48	Emisión local, credenciales, grant, scopes, consumidor y expiración.
`01 - Scope matrix - 30 combinations`	30	Scope individual contra seis operaciones y rechazo previo a mutaciones.
`01b - All 32 scope combinations - 192 flows`	192	Matriz parametrizada completa, incluidas combinaciones vacías.
`02 - Authentication and resource visibility`	42	400/401/403/404 y no filtración de recursos ajenos.

Carpeta de la colección	Solicitudes actuales	Pruebas derivadas
`03 - accounts.read`	5	Lista, vacío, detalle y respuestas incompletas sin datos parciales.
`04 - balances.read - includes business errors HTTP 200`	13	Balances positivos, cero, negativos, retenciones y excepción 677–680.
`05 - Upstream and infrastructure scenarios per scope`	32	500/502/503/504, JSON corrupto, escenario inválido y 429 controlado.
`06 - accounts.write`	12	Cinco productos, validación, mapping no disponible y conflicto.
`07 - balances.adjust`	21	Crédito/débito, compensación, precisión, fondos, UUID y replay/Processing.
`08 - accounts.close - run last`	14	Matriz de estados, precondiciones, cierre repetido y cuenta recién creada.

Un caso puede producir varias pruebas o filas parametrizadas; el número de tests no tiene por qué ser 409. Se asignará un ID estable a cada escenario y se indicará qué suite verifica cada condición. Para evitar depender del orden de Postman, cada prueba preparará su estado; los cierres no contaminarán las consultas posteriores.

8.9.2 Niveles de prueba

Suite	Alcance	Dependencias y aserciones
UnitTests	Domain y casos de uso Application.	Sin servidor web, Docker ni SQL; dobles de puertos y reloj controlable. Invariantes, transiciones, normalización, precedencia y errores.
FunctionalTests	Pipeline HTTP completo con `WebApplicationFactory/Microsoft.AspNetCore.Mvc.Testing`.	FastEndpoints, binding, validators, JWT real de taller, políticas, scopes, RFC 9457, excepción 200 y OpenAPI; puertos externos controlados.
IntegrationTests	Adaptadores, SQL Server real y topología local.	Contenedores/bases aisladas, EF, migraciones, seed, constraints, locks, transacciones, mock HTTP y arranque de AppHost con `Aspire.Hosting.Testing`.

Las pruebas de arquitectura pueden residir en UnitTests y verifican referencias/prohibiciones. Los tests HTTP con dobles no demuestran atomicidad SQL; SQL InMemory o SQLite no sustituyen a SQL Server para bloqueos, precisión, `rowversion` y concurrencia. No se invocará Mambu real.

Se elegirán framework y runner compatibles con .NET 10 y Aspire, centralizando sus paquetes. Antes de fijar comandos `dotnet test` se verificará si se utiliza VSTest o Microsoft.Testing.Platform: sus opciones de filtros/reportes no son intercambiables.

8.9.3 Ciclo obligatorio Red/Green/Refactor

1. Escribir primero la prueba de la regla o cambio de transporte; comprobar que se descubre y falla por la razón esperada, no por una conexión o compilación rota.
2. Implementar el mínimo código necesario en la capa propietaria y ejecutar de nuevo esa prueba.
3. Refactorizar manteniendo verde el test y ejecutar la regresión del incremento.
4. Conservar IDs, comando, resultado esperado/obtenido y revisión, sin secretos.

Las pruebas de caracterización de comportamiento ya existente pueden nacer verdes: capturan el legado antes de tocarlo, pero no se presentarán como evidencia Red de una funcionalidad nueva. Antes de cambiar el comportamiento o extraer la implementación, se añadirá el test discriminante que falta. No se borrarán o relajarán aserciones para aceptar una regresión.

8.9.4 Riesgos y casos límite adicionales

- Dos ajustes simultáneos con la misma clave y payload: un único movimiento; respuesta coherente de procesamiento/replay y sin doble débito.
- Misma clave con distinto importe, cuenta, moneda o `reason`: 409; distinto consumidor mantiene el aislamiento de la clave.
- Dos retiros concurrentes que excederían fondos combinados: sin saldo inválido ni posting parcial. Cierre y ajuste concurrentes respetan invariantes.
- Fallo entre actualización de balances, movimiento y registro idempotente: rollback íntegro; comportamiento definido ante cancelación.
- Replay de un error con otra correlación: código y resultado estable conservados, metadata actual coherente y referencia a correlación original.
- Cuentas `acc-50677` a `acc-50680`, `acc-50006`, `acc-50007` y `acc-50008`: HTTP 200 y cuerpo esperado, incluso tras adoptar Problem Details.
- Conflictos combinados entre falta de token/scope, recurso ajeno y balance bloqueado: el control de acceso se resuelve primero.
- Migración sobre base vacía y existente, segunda ejecución sin duplicar seed y fallo del migrador sin arrancar la API.
- Host sin declaraciones: ningún servicio FastEndpoints ni documento/UI Swagger; host con declaraciones: rutas y documento correctos.
- Saturación 429 con límites y ventana controlados en una prueba aislada; no depender de repetir manualmente llamadas hasta que falle.

8.10 Paso 7. Revisar, validar y aprobar

Se revisarán por merge request los artefactos Spec Kit, cambios de código, pruebas, documentación y evidencias. La aprobación exige una comparación contra la línea base y una explicación de cada diferencia autorizada, especialmente la metadata de correlación.

La matriz final relacionará **requisito → escenario Postman → prueba → capa → resultado → revisión**. Para cada antipatrón figurará corregido y demostrado o, únicamente en el caso 5, preservado por excepción constitucional. No se declarará «corregidos todos» ocultando esa excepción.

Se actualizarán README, instrucciones de arranque y documentación OpenAPI para describir la implementación resultante, manteniendo el contrato original como referencia histórica salvo un cambio contractual aprobado. La guía rápida deberá distinguir comandos que ya existen de los incorporados durante el taller.

9 Validación y entrega con Jenkins

Jenkins será el orquestador propuesto para automatizar las comprobaciones; la definición de pipeline se mantendrá en el repositorio de automatización acordado. La ejecución local y Jenkins deberán usar SDK, paquetes y herramientas compatibles y fijados.

Etapa	Comprobación	Evidencia
1. Recuperar fuentes y gobierno	Revisión exacta, constitución coherente y referencias compartidas necesarias disponibles.	Identidad de fuentes y configuración efectiva.
2. Restaurar	SDK .NET 10, catálogo CPM, fuentes y lockfiles según política.	Restauración reproducible; fallo ante versiones locales/overrides.
3. Compilar y revisar arquitectura	Solución Release, referencias hacia dentro y ausencia de MVC/Minimal APIs de operaciones.	Build y controles automatizados.
4. Ejecutar unitarias	Invariantes y casos de uso sin infraestructura.	Resultados y evidencia TDD vinculada a las tareas.
5. Ejecutar funcionales	Pipeline HTTP, seguridad, errores, compatibilidad y host vacío.	Resultados por requisito y escenario.
6. Ejecutar integración	SQL real, migraciones, transacciones, concurrencia y Aspire.	Resultados, logs seguros y recursos aislados identificados.
7. Validar OpenAPI y regresión	Documento completo y conforme al comportamiento aprobado; Newman como comprobación complementaria.	Diferencias clasificadas y formatos HTTP comprobados.
8. Construir y conservar entrega	Imagen .NET 10 no root, artefactos e informes ligados a la revisión.	Versión, revisión, huella `sha256` y evidencias aun si falla una etapa.

REGLA DE ORO DE LA PIPELINE

La especificación aprobada, la constitución y las pruebas gobiernan la evolución; una compilación correcta o un informe Newman sin diferencias no demuestran por sí solos que la modernización esté completa.

- No se desactivarán validaciones para conseguir verde ni se omitirán suites de infraestructura por conveniencia.
- Spectral y Redocly podrán validar el OpenAPI si se adoptan con reglas/versiones fijadas; no sustituyen tests de ejecución, seguridad o atomicidad.
- Los casos excluidos y sus motivos serán visibles. El 429 que antes era manual deberá tener una prueba automatizada controlada.
- La nueva regresión contra Aspire utilizará su dirección real y una base aislada. El script Compose actual no se reutilizará sin adaptar y verificar su ciclo de vida.
- No se destruirán datos o servicios ajenos. La limpieza solo afectará recursos de prueba identificados y autorizados; los informes no incluirán secretos.
- Generar imagen o artefactos no autoriza publicar la API ni desplegarla en producción.

10 Criterios de aceptación

- La especificación cumple las 7 C, diferencia legado/objetivo/excepciones y no deja decisiones relevantes sin aprobar.
- La constitución efectiva permite implementación brownfield y contiene expresamente el bug HTTP 200 aceptado, el TDD obligatorio y el principio VII de paquetes/API Standards.
- El backend está en C# y .NET 10 con SDK fijado; todas las versiones NuGet están en el catálogo central y los proyectos no declaran versiones ni overrides.
- Domain no depende de ASP.NET Core, EF, HTTP, Application o Infrastructure; Application no depende de Infrastructure/Api y existen controles automatizados de estos límites.
- Se registran servicios por capa; las reglas financieras e invariantes no residen en endpoints o persistencia.
- Todas las operaciones HTTP, incluidos token mock y salud, utilizan FastEndpoints 8.3.0 con clase propia, `Configure()` y `HandleAsync` con cancelación propagada; no quedan controllers o mappings Minimal API de operaciones.
- FastEndpoints.Swagger 8.3.0 usa `SwaggerDocument(...)` y `UseSwaggerGen()`; sin declaraciones no se registra FastEndpoints ni se define/expone Swagger, y ambos hosts están probados.
- Aspire reproduce SQL persistente, secretos, conexiones y el orden SQL saludable → migrador terminado → API; un fallo de migración bloquea el arranque.
- Se verifica compatibilidad con una base existente y una vacía, sin perder datos, duplicar seed o sustituir migraciones por `EnsureCreated`.
- Se preservan importes decimales, precisión, fondos, sobregiro, estados, retenciones, auditoría, idempotencia y atomicidad bajo concurrencia.
- Existen suites unitarias, funcionales e integración con trazabilidad a la colección Postman y evidencia de pruebas anteriores a cada implementación nueva.
- Se cubren las 32 combinaciones de scopes, visibilidad, OAuth mock, errores upstream y un 429 automatizado y aislado.
- Los errores HTTP utilizan RFC 9457 y estados apropiados; los errores OAuth mantienen su contrato de protocolo documentado.
- **677–680 siguen bajo HTTP 200, `application/json`, payload y mensajes originales, sin Problem Details ni balances parciales; las pruebas también verifican prioridad y autorización previa.**
- El replay corrige la correlación según la decisión aprobada sin cambiar el resultado estable ni generar un segundo movimiento.
- Los antipatrones 1, 2, 3, 4, 6 y 7 están corregidos con evidencias; el 5 está documentado y protegido como excepción.
- OpenAPI refleja operaciones, scopes, DTOs, validaciones, errores y variantes del 200 de balances, sin documentación parcial deliberada.
- Compilación, suites, validaciones y regresión satisfacen las puertas acordadas; Jenkins conserva evidencias por revisión sin secretos.
- README y guía de arranque describen el resultado real, Compose y Aspire no colisionan y no se ha conectado Mambu real ni desplegado en producción.

11 Resultado esperado

CONCLUSIÓN Y ENTREGABLE FINAL

Al finalizar el taller, el equipo dispondrá de una evolución de Core Banking API especificada y trazable, ejecutable sobre .NET 10, organizada con Clean Architecture, con NuGet centralizado, FastEndpoints 8.3.0 y Aspire para desarrollo local. La solución estará protegida por pruebas unitarias, funcionales e integración

creadas antes de cada implementación nueva y derivadas de los escenarios existentes.

El resultado corrige las deficiencias estructurales, completa OpenAPI y estandariza los errores HTTP con RFC 9457, pero conserva explícitamente el bug de balances bajo HTTP 200 como comportamiento aceptado. El paso a una integración real con Mambu o a una corrección futura de esa excepción será una iniciativa independiente.

12 Referencias oficiales y uso del MCP

Durante el taller se utilizará primero el MCP de documentación Microsoft para buscar y recuperar las páginas relevantes. Si no está disponible, se registrará la limitación y se consultarán directamente las fuentes oficiales; no se afirmará haber ejecutado una consulta MCP inexistente. Para preparar este guion no se obtuvo acceso operativo a ese MCP y se consultaron fuentes web oficiales. Los comandos y APIs se verificarán de nuevo frente a las versiones fijadas al ejecutar el taller.

- Microsoft: Clean Architecture y organización de proyectos.
- Microsoft: principios arquitectónicos.
- Microsoft: NuGet Central Package Management y sus reglas.
- Microsoft: manejo de errores y Problem Details en ASP.NET Core 10.
- Microsoft: pruebas de integración de ASP.NET Core.
- Microsoft: aplicación de migraciones EF Core.
- Aspire: SQL Server en AppHost, imagen, volumen y conexiones.
- FastEndpoints: operaciones y configuración.
- FastEndpoints: soporte Swagger.
- NuGet: FastEndpoints 8.3.0.
- RFC 9457: Problem Details for HTTP APIs.
- Spec Kit: repositorio y flujos oficiales.
- Spec Kit: incorporación a proyectos existentes.